

NTT-BY Interview: 01OCT2026

Senior Java Backend Interview Notes - 9/10 Years Experience

Purpose

This document keeps the original interview questions and adds senior-level answers. For a 9-10 year Java backend profile, do not stop at definitions. Explain:

- how the feature works,
- where you used or would use it,
- failure scenarios,
- performance/scalability implications,
- trade-offs and alternatives.

IMPORTANT FOR THE INTERVIEW

Do not claim that you used a technology in production unless you actually used it. Where this document says "example answer", adapt the project-specific part to your real experience.

Q1) How to implement Rollback in java application, consider @transactional annotation not present in real time?

What the interviewer is testing

They want to know whether you understand transaction management beyond simply putting `@Transactional` on a method.

Senior-level short answer

"`@Transactional` is Spring's declarative transaction management. If I cannot use it, I can manage the transaction programmatically. In Spring I would normally use `TransactionTemplate` or `PlatformTransactionManager`. With plain JDBC I can disable auto-commit and explicitly call `commit()` or `rollback()`. If the operation spans multiple independent databases or microservices, a local transaction is not sufficient; then I would evaluate XA/JTA for strict atomicity or, more commonly in microservices, a Saga with compensating transactions, idempotency, retries, and persistent workflow state."

Scenario A - Plain JDBC transaction

Example:

```
Connection connection = dataSource.getConnection();

try {
    connection.setAutoCommit(false);

    // DB operation 1
    // DB operation 2
```

```
// DB operation 3

connection.commit();

} catch (Exception ex) {
    connection.rollback();
    throw ex;

} finally {
    connection.close();
}
```

Key point:

- setAutoCommit(false) starts manual transaction control.
- commit() permanently saves the changes.
- rollback() reverts the uncommitted changes.
- Always release the connection, normally via try-with-resources.

Scenario B - Spring TransactionTemplate (preferred programmatic Spring approach)

Example:

```
@Service
public class OrderService {

    private final TransactionTemplate transactionTemplate;

    public OrderService(PlatformTransactionManager transactionManager) {
        this.transactionTemplate = new TransactionTemplate(transactionManager);
    }

    public void createOrder() {
        transactionTemplate.executeWithoutResult(status -> {
            try {
                saveOrder();
                saveOrderItems();
                updateInventory();
            } catch (Exception ex) {
                status.setRollbackOnly();
                throw ex;
            }
        });
    }
}
```

Why TransactionTemplate is useful:

- Explicit transaction boundary.
- Easier to reason about than manual begin/commit/rollback calls.
- Uses Spring's PlatformTransactionManager.
- Useful when transaction boundaries must be decided dynamically.

Scenario C - PlatformTransactionManager directly

```
TransactionStatus status = transactionManager.getTransaction(
    new DefaultTransactionDefinition());

try {
    saveOrder();
    savePayment();
}
```

```

        transactionManager.commit(status);

    } catch (Exception ex) {
        transactionManager.rollback(status);
        throw ex;
    }

```

Use this when you need very explicit control over transaction creation and completion. For most Spring business code, `TransactionTemplate` is cleaner.

Senior follow-up: What if DB1 succeeds and DB2 fails?

First clarify whether DB1 and DB2 participate in the same transaction manager.

Case 1 - Same database / same local transaction

If both operations are in the same database transaction and neither has committed yet, a rollback can revert both operations.

Case 2 - Two independent databases

If DB1 has already committed and DB2 then fails, a normal local transaction cannot magically undo the DB1 commit.

There are two major strategies:

A) Saga pattern - common in microservices

Example business flow:

```

Order Service      -> DB1
Payment Service    -> DB2
Inventory Service  -> DB3

```

Forward flow:

1. Create order -> SUCCESS
2. Charge payment -> SUCCESS
3. Reserve inventory -> FAILURE

The first two local transactions are already committed. Instead of a database rollback, perform business compensations:

1. Refund payment
2. Cancel order

Final state:

- Order = CANCELLED
- Payment = REFUNDED
- Inventory = NOT_RESERVED

Important distinction: A compensating transaction is not necessarily the physical reverse SQL statement. It is a business operation that brings the system to a consistent state.

Why idempotency is required

Suppose the refund message is delivered twice because of retry:

```
refund(paymentId=1001) refund(paymentId=1001)
```

The service must not refund twice.

Conceptually:

```
if (payment.status() == REFUNDED) {
    return; // already processed
}
```

```
refund(payment);
```

A real implementation may also store an idempotency key / processed event ID.

Why retry is required

Compensation can also fail temporarily.

Example:

- Inventory reservation fails.
- Refund request is sent.
- Payment service is temporarily unavailable.
- Retry later using exponential backoff.

Retries should be:

- bounded,
- backoff-based,
- used only for retryable/transient failures,
- combined with idempotency.

Why persistent Saga state is required

Do not keep the entire workflow only in memory. If the service crashes, it must know which steps were completed.

Example state:

SAGA_ID	STEP	STATUS
123	CREATE_ORDER	SUCCESS
123	TAKE_PAYMENT	SUCCESS
123	RESERVE_INVENTORY	FAILED
123	REFUND_PAYMENT	PENDING
123	CANCEL_ORDER	PENDING

After restart, processing can resume from the persisted state.

Saga styles

1. Choreography

OrderCreated event

-> Payment Service

PaymentCompleted event

-> Inventory Service

InventoryReservationFailed event

-> Payment refund / Order cancellation

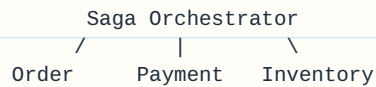
Advantages:

- Loose coupling.
- No central orchestrator.

Disadvantages:

- Complex event chains become harder to understand and debug.

2. Orchestration



The orchestrator decides the next step and compensation.

Advantages:

- Easier to visualize complex workflows.
- Centralized workflow state/decision logic.

Disadvantages:

- Orchestrator itself becomes an important component to design and operate correctly.

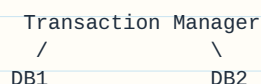
Production additions for Saga

- Transactional Outbox to avoid DB-update + message-publish inconsistency.
- Idempotent consumers.
- Retry with backoff.
- Dead-letter handling.
- Correlation/Saga ID.
- Audit trail.
- Metrics and distributed tracing.
- Timeouts for stuck workflows.

B) XA/JTA and Two-Phase Commit (2PC)

Goal: Either all participating resources commit or all roll back as one distributed transaction.

Conceptual topology:



Phase 1 - PREPARE

The transaction manager asks every participant: "Are you able to commit?"

Example:

- DB1 -> YES
- DB2 -> YES

The resources prepare the transaction but do not complete the final commit yet.

Phase 2 - COMMIT

If every participant prepared successfully:

- DB1 -> COMMIT
- DB2 -> COMMIT

If a participant cannot prepare:

- coordinator instructs rollback.

Why XA/JTA is not automatically provided by a normal @Transactional

A normal local @Transactional usually operates through one transaction manager/resource. To coordinate multiple XA-capable resources, you need an XA/JTA-capable transaction manager and compatible resources/drivers.

Saga vs XA/JTA

Saga:

- Multiple local transactions.
- Failure handled by compensation.
- Usually eventual consistency.
- Better fit for loosely coupled microservices.
- Higher business-workflow complexity.

XA/JTA 2PC:

- Coordinated distributed transaction.
- Attempts atomic commit/rollback across resources.
- Stronger consistency.
- More infrastructure coupling.
- Can hold resources longer and reduce availability/scalability.

Interview one-liner: "Saga means commit local transactions and compensate later if a downstream step fails. XA/2PC means do not complete the distributed commit until all participating resources agree that they can commit."

Important senior follow-up: What if a message must be published after DB commit?

A classic failure window is:

1. update database succeeds,
2. application crashes before publishing Kafka/message event.

Use Transactional Outbox:

- Business data and outbox event are stored in the same local DB transaction.
- A separate publisher/CDC process sends the outbox event.
- Consumer is idempotent.

This avoids trying to make the database update and message broker send one fragile application-level operation.

Q2) Spring: Primary vs Qualifier annotation?

Problem

Suppose two beans implement the same interface:

```
public interface PaymentService { }

@Service
@Primary
class UpiPaymentService implements PaymentService { }

@Service
class CardPaymentService implements PaymentService { }
```

@Primary

@Primary tells Spring which bean should be the default candidate when multiple beans of the same type exist.

Example:

```
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

Because UpiPaymentService is @Primary, it will be selected by default.

@Qualifier

@Qualifier explicitly selects a bean at an injection point.

Example:

```
public OrderService(
    @Qualifier("cardPaymentService") PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

This injects CardPaymentService even though UpiPaymentService is @Primary.

Senior-level answer

"@Primary defines the default bean when multiple candidates of the same type are available.

@Qualifier makes the dependency explicit and selects a specific candidate. A matching qualifier is more specific than the generic primary candidate. For required dependencies I prefer constructor injection, and I use @Qualifier when different business implementations are intentionally needed."

Remember:

```
@Primary    = default choice
@Qualifier  = explicit choice
```

Follow-ups

1. What if two beans are both @Primary?

Spring can still face ambiguity; do not configure multiple primaries for the same injection scenario unless another discriminator resolves it.

2. Can @Qualifier be used with constructor injection?

Yes, and constructor injection is normally preferred for required dependencies.

3. Can custom qualifier annotations be created?

Yes. For a large codebase, custom qualifier annotations can be clearer than many string bean names.

Q3) Java 8 to 2X latest features? Which one you used?

Important current context (October 2026)

- Java 27 is the latest Java SE release.
- Java 25 is the latest LTS release.
- Common enterprise LTS generations are Java 8, 11, 17, 21, and 25.

For a senior interview, do not try to recite every JEP. Group the important changes and go deeper on features you actually used.

Java 8

- Lambda expressions.
- Functional interfaces.
- Stream API.
- Optional.
- Method references.
- Default/static methods in interfaces.
- java.time API.
- CompletableFuture.

Example:

```
List<String> activeNames = employees.stream()
    .filter(Employee::isActive)
    .map(Employee::getName)
    .toList(); // toList itself is from a later Java release
```

Java 9

- Java Platform Module System (JPMS).
- JShell.
- List.of(), Set.of(), Map.of().
- Private methods in interfaces.

Java 10

- Local-variable type inference using var.

```
var employees = new ArrayList<Employee>();
```


Important: var is compile-time type inference, not dynamic typing.

Java 11 LTS

- Standard HttpClient.
- String.isBlank(), strip(), lines(), repeat().
- Files.readString()/writeString().
- var in lambda parameters.

Java 12-16 (important highlights)

- Switch expressions evolved and became standard.
- Helpful NullPointerException diagnostics.
- Text blocks.
- Records.
- Pattern matching for instanceof.
- Stream.toList().

Record example:

```
public record OrderResponse(Long id, String status) { }
```

Java 17 LTS

- Sealed classes/interfaces became standard.
- Stronger encapsulation of JDK internals.
- Mature modern Java baseline for many enterprise applications.

Example:

```
public sealed interface Payment
    permits CardPayment, UpiPayment { }
```

Java 18-20 (important direction)

- UTF-8 by default.
- Simple web server.
- Virtual Threads previewed.
- Record patterns previewed.
- Structured Concurrency incubated.

Java 21 LTS

Very important for current Java backend interviews:

- Virtual Threads became final.
- Record Patterns.
- Pattern Matching for switch.
- Sequenced Collections.

Example:

```
String result = switch (payment) {
    case CardPayment card -> "CARD";
    case UpiPayment upi    -> "UPI";
};
```

Java 22-27 - what to say at interview level

Recent releases continue improvements in:

- concurrency and context propagation,
- pattern matching,
- native interoperability / Foreign Function & Memory API,
- stream processing extensions,
- class-file APIs,
- runtime/performance/security,
- simpler source-file and language ergonomics.

Do not claim detailed production usage of a preview feature unless you really used it. Always distinguish FINAL vs PREVIEW/INCUBATOR features for the Java version being discussed.

Suggested answer for "Which features have you used?"

Adapt this to your actual experience:

"In production I have used Java 8 features extensively - Streams, lambdas, functional interfaces, Optional, java.time, and CompletableFuture. In newer code I have worked with Java 17/21 style features such as records, text blocks, modern switch/pattern matching, and newer API improvements. I understand Virtual Threads in Java 21 and would use them for high-concurrency blocking I/O workloads. I would not claim that every latest Java feature has been used in production; I separate what I used from what I have evaluated."

Senior follow-ups to prepare

- map() vs flatMap().
- Intermediate vs terminal stream operations.
- Why parallelStream() can be dangerous in server applications.
- Optional best practices.
- Record vs normal class.
- Sealed class use cases.
- Virtual Thread vs CompletableFuture vs WebFlux.
- Migration challenges from Java 8/11 to 17/21/25.

Q4) Virtual thread?

Senior-level answer

"A Virtual Thread is a lightweight Java Thread implementation designed to support a very large number of concurrent tasks. It is managed by the JVM and is not permanently mapped one-to-one to an OS thread. Virtual Threads became a final feature in Java 21 under Project Loom. They are especially useful for applications that have many blocking I/O operations, because we can keep the simple thread-per-request programming model without requiring one expensive platform thread per request."

Example:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++) {
        executor.submit(() -> callExternalService());
    }
}
```

Traditional model (simplified):

Request -> Platform Thread -> OS Thread

Virtual-thread model (simplified):

Many Virtual Threads -> JVM scheduling -> smaller set of carrier/platform threads

Best use cases

- REST calls.
- Database calls.
- Network I/O.
- File I/O.
- High-concurrency request processing with blocking APIs.

Not a magic solution for CPU-bound work

If 1,000 tasks all need heavy CPU, the machine still has a limited number of CPU cores. Virtual Threads do not make CPU computation itself faster.

Virtual Thread vs traditional thread pool

Platform threads are expensive enough that we normally use bounded pools. Virtual Threads are cheap enough that the common model is one virtual thread per task, while concurrency to scarce downstream resources should be controlled separately.

Example: Do not allow 100,000 virtual threads to open 100,000 DB connections. The DB connection pool/semaphore still limits database concurrency.

Virtual Thread vs CompletableFuture

Virtual Threads:

- Keep imperative/sequential-looking code.
- Great for blocking I/O concurrency.
- Easier stack traces/debugging than deeply chained async code in many cases.

CompletableFuture:

- Good for asynchronous composition and combining independent computations.

- Can run multiple calls and combine results.
- Requires careful executor and exception handling.

Virtual Thread vs WebFlux/reactive

Virtual Threads allow blocking-style programming at high concurrency. Reactive/WebFlux uses non-blocking/event-driven processing and backpressure. Do not migrate to one solely because it is newer; choose based on workload, ecosystem, and operational requirements.

Potential follow-ups

- What is a carrier thread?
- Is ThreadLocal supported? Yes, but large-scale usage deserves care.
- Should I create a fixed-size pool of Virtual Threads? Usually no; use one per task and control actual scarce resources separately.
- What is pinning? In older Loom implementations/JDKs, some blocking while holding certain monitors/native frames could pin a virtual thread to its carrier. Newer JDKs have continued to improve this area, so answer according to the deployed JDK version.

Q5) CompletableFuture what is this? When should it used?

Definition

CompletableFuture is Java's API for representing, executing, composing, and handling asynchronous computations.

Senior-level answer

"I use CompletableFuture when independent operations can execute concurrently and I need to compose their results without manually managing Thread objects. For example, if I need customer information, order history, and recommendations from independent services, I can call them concurrently and combine the results. I also pay attention to which Executor is used, timeouts, exception handling, cancellation, and not blocking the common ForkJoinPool with unsuitable workloads."

Example - two independent calls

```
CompletableFuture<User> userFuture =
    CompletableFuture.supplyAsync(this::getUser, executor);

CompletableFuture<List<Order>> orderFuture =
    CompletableFuture.supplyAsync(this::getOrders, executor);

CompletableFuture<UserResponse> responseFuture =
    userFuture.thenCombine(
        orderFuture,
        UserResponse::new
    );
```

If getUser takes 2 seconds and getOrders takes 3 seconds, sequential execution may take about 5 seconds, while concurrent execution may complete in around the slower call's latency plus overhead, assuming they are truly independent and downstream capacity exists.

Important methods

- `supplyAsync()` -> asynchronous task returning a value.
- `runAsync()` -> asynchronous task with no return value.
- `thenApply()` -> transform a result.
- `thenCompose()` -> compose dependent async operations; avoids nested future.
- `thenCombine()` -> combine two independent futures.
- `allOf()` -> wait/combine completion of many futures.
- `anyOf()` -> continue when any future completes.
- `exceptionally()` -> recover from exception.
- `handle()` -> handle both success and failure.
- `orTimeout()` -> timeout support.
- `completeOnTimeout()` -> default value after timeout.

thenApply vs thenCompose

If function returns a normal value: `thenApply()`

If function returns another `CompletableFuture`: `thenCompose()`

Bad shape:

```
CompletableFuture<CompletableFuture<Result>>
```

Use `thenCompose()` to flatten it to:

```
CompletableFuture<Result>
```

Executor concern

`CompletableFuture.supplyAsync(task)` without an explicit executor commonly uses the

`ForkJoin` common pool.

For production service workloads, a dedicated executor is often safer when you need control over:

- pool size,
- queue size,
- isolation,
- naming,
- monitoring,
- rejection policy.

Common mistake

Calling `future.get()` immediately after every `supplyAsync()` can destroy the benefit of asynchronous composition by blocking too early.

Q6) How to thread communications in modern way? ThreadWare?

Note on wording

"ThreadWare" is not a standard modern Java concurrency API name that I can reliably map to this question. The interviewer most likely meant modern inter-thread communication / thread-aware concurrency mechanisms. If they meant a specific internal term, ask them to clarify it.

Traditional low-level mechanisms

- synchronized.
- wait().
- notify().
- notifyAll().

They still work, but senior application code usually prefers higher-level concurrency utilities because they are easier to reason about and less error-prone.

Modern options

1. BlockingQueue - Producer/Consumer

```
BlockingQueue<String> queue = new LinkedBlockingQueue<>();

// Producer
queue.put("event");

// Consumer
String event = queue.take();
```

Benefits:

- No manual wait()/notify().
- Clear producer-consumer semantics.
- Can use bounded queues for backpressure.

2. CompletableFuture

Useful when one asynchronous stage needs to notify/trigger another stage or when results must be combined.

3. CountdownLatch

One or more threads wait until a fixed number of operations complete.

4. CyclicBarrier / Phaser

Useful when multiple threads/tasks need to meet at synchronization phases.

5. Semaphore

Limits concurrent access to a scarce resource.

Example: Only 20 concurrent requests should call a slow third-party API.

6. Concurrent collections

- ConcurrentHashMap.
- CopyOnWriteArrayList (read-heavy special case).
- ConcurrentLinkedQueue.

7. Locks and Conditions

ReentrantLock + Condition provides more flexible explicit locking than synchronized in some cases.

8. Message/event based communication

Inside distributed systems, do not use Java thread primitives across services. Use Kafka/RabbitMQ/events/queues where appropriate.

Senior answer

"For modern Java I avoid manual wait/notify unless the problem genuinely needs monitor-level control. I prefer higher-level java.util.concurrent abstractions such as BlockingQueue for producer-consumer, CompletableFuture for async composition, CountdownLatch/Phaser for coordination, Semaphore for concurrency limits, and concurrent collections for shared state. Between microservices, thread communication is not the model; I use messaging/events or synchronous APIs depending on the requirement."

Q7) MaxLimit of threadpool/task limit?

Key interview point

There is no universal "maximum thread pool size" number for Java applications. The right size depends on:

- CPU cores.
- CPU-bound vs I/O-bound workload.
- memory per thread.
- task latency.
- downstream capacity.
- connection pools.
- queue policy.
- operating system/container limits.
- SLA and backpressure strategy.

CPU-bound tasks

A common starting point is around: NCPU or NCPU + 1

Example:

8 CPU cores -> start testing around 8-9 active CPU worker threads.

Adding 100 CPU workers to an 8-core machine usually increases context switching rather than useful throughput.

I/O-bound tasks

More threads can be useful because tasks spend much of the time waiting. A sizing approximation sometimes used is:

$$\text{Threads} \sim \text{NCPU} * (1 + \text{WaitTime} / \text{ComputeTime})$$

Example: NCPU = 8 WaitTime = 80 ms ComputeTime = 20 ms

$$\begin{aligned} \text{Threads} &\sim 8 * (1 + 80/20) \\ &\sim 40 \end{aligned}$$

This is only a starting estimate. Measure under realistic load.

ThreadPoolExecutor example

```
ThreadPoolExecutor executor = new ThreadPoolExecutor(
    10,                      // corePoolSize
    20,                      // maximumPoolSize
    60,
    TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(1000),
    new ThreadPoolExecutor.CallerRunsPolicy()
);
```

Senior point: queue size matters as much as thread count.

A very large/unbounded queue can:

- hide overload,
- increase latency,
- consume memory,
- eventually cause OOM or long request delays.

Use:

- bounded queue,
- appropriate rejection/backpressure strategy,
- metrics for active threads/queue depth/rejected tasks,
- downstream protection.

Rejection policies

- AbortPolicy.
- CallerRunsPolicy.
- DiscardPolicy.
- DiscardOldestPolicy.
- custom RejectedExecutionHandler.

Executors helper warning

Convenience factory methods are easy to use, but always understand their queue/thread semantics. For production systems where overload behavior matters, an explicitly configured ThreadPoolExecutor is often clearer.

Virtual Threads change the model

With Virtual Threads, you normally do not size a worker pool the same way as platform threads. You can create a virtual thread per task, but you STILL must limit access to scarce downstream resources such as:

- DB connections,
- external APIs,
- rate-limited services,
- memory-heavy work.

Q8) How we can get application context in a springboot application?

Best answer first

Normally, application business code should not fetch dependencies from `ApplicationContext`. Prefer constructor dependency injection.

Preferred:

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

If you genuinely need `ApplicationContext`

Option 1 - inject it

```
@Component
public class SomeComponent {

    private final ApplicationContext applicationContext;

    public SomeComponent(ApplicationContext applicationContext) {
        this.applicationContext = applicationContext;
    }
}
```

Option 2 - `ApplicationContextAware`

```
@Component
public class ApplicationContextProvider
    implements ApplicationContextAware {

    private static ApplicationContext context;

    @Override
    public void setApplicationContext(ApplicationContext applicationContext) {
        context = applicationContext;
    }

    public static ApplicationContext getContext() {
```

```

        return context;
    }
}

```

Then: `PaymentService service =`

```
ApplicationContextProvider.getContext().getBean(PaymentService.class);
```

Senior caveat

A static `ApplicationContext` holder/service-locator pattern increases coupling to Spring and makes tests/design harder. Use it only for justified infrastructure/integration cases, not as the default way to obtain dependencies.

ApplicationContext from main method

`ConfigurableApplicationContext context =`

```
SpringApplication.run(Application.class, args);
```

```
PaymentService bean = context.getBean(PaymentService.class);
```

Interview answer

"We can inject `ApplicationContext`, implement `ApplicationContextAware`, or get the `ConfigurableApplicationContext` returned by `SpringApplication.run()`. But in normal

business services I prefer constructor injection and avoid manual `getBean()` calls because

otherwise the code becomes a service-locator style and is unnecessarily coupled to the Spring container."

Q9) Spring bean life cycle?

Senior-level lifecycle

1. Bean definitions are discovered/registered.
2. `BeanFactoryPostProcessor` can modify bean definitions before normal bean instantiation.
3. Spring instantiates the bean (constructor/factory method).
4. Dependencies/properties are populated.
5. Aware callbacks are invoked where applicable.
6. `BeanPostProcessor` before-initialization callbacks run.
- `@PostConstruct` is normally invoked through Spring post-processing infrastructure during this initialization phase.
7. Initialization callbacks run:
 - `InitializingBean.afterPropertiesSet()`
 - custom init method (if configured)
8. `BeanPostProcessor` after-initialization callbacks run.
 - infrastructure can wrap the bean with proxies, for example AOP-related behavior.
9. Bean is ready for normal use.
10. On `ApplicationContext` shutdown, destruction callbacks for managed singleton beans run:

- @PreDestroy
- DisposableBean.destroy()
- custom destroy method.

Simplified flow

Bean Definition

- > BeanFactoryPostProcessor
- > Instantiation
- > Dependency Injection
- > Aware callbacks
- > BeanPostProcessor before initialization
 - > @PostConstruct handled in this phase
- > afterPropertiesSet()/custom init
- > BeanPostProcessor after initialization
 - > proxy may be returned
- > Bean ready
- > Context shutdown
 - > @PreDestroy / destroy()

Aware interfaces examples

- BeanNameAware.
- BeanFactoryAware.
- ApplicationContextAware.

Why BeanPostProcessor is important

Spring infrastructure uses bean post-processing to implement many framework behaviors. A bean reference seen by another component may be a proxy instead of the raw object. This is relevant for concepts such as:

- AOP,
- transactions,
- async execution,
- caching,
- security interception.

Prototype-scope caveat

Spring creates and initializes prototype beans, but it does not manage their complete lifecycle through normal destruction in the same way as singleton beans. Cleanup of prototype resources is normally the caller/application's responsibility.

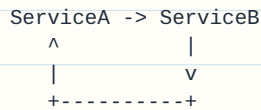
30-second interview answer

"Spring first creates the bean from its definition, performs dependency injection, invokes Aware callbacks, and passes it through BeanPostProcessors. Initialization callbacks such as @PostConstruct and afterPropertiesSet are invoked during initialization, and post- processing after initialization may return an AOP/proxy-wrapped bean. The bean is then ready for use. On context shutdown Spring invokes destruction callbacks such as

@PreDestroy for managed singleton beans. I also distinguish BeanFactoryPostProcessor, which works on bean definitions, from BeanPostProcessor, which works on bean instances."

Q10) Circular dependency in Spring, how to avoid it? 3 ways?

Example problem



Constructor cycle:

```

@Service
class ServiceA {
    ServiceA(ServiceB serviceB) { }
}

@Service
class ServiceB {
    ServiceB(ServiceA serviceA) { }
}

```

With constructor injection, Spring cannot construct A without B and cannot construct B without A.

Best solution 1 - Redesign responsibilities

Circular dependency often signals that two services know too much about each other.

Before:

```
A -> B -> A
```

After:



Extract the shared responsibility into another component. This is the preferred senior-level answer.

Solution 2 - Event-driven decoupling

Instead of A directly calling B and B directly calling A:

```
A -> publish event -> B handles event
```

Options:

- Spring ApplicationEvent for in-process decoupling.
- Kafka/RabbitMQ for distributed asynchronous workflows.

Use this only when asynchronous/event semantics are appropriate.

Solution 3 - @Lazy

```

@Service
class ServiceA {

    private final ServiceB serviceB;

    ServiceA(@Lazy ServiceB serviceB) {

```

```

        this.serviceB = serviceB;
    }
}

```

Spring injects a lazy proxy and delays resolution.

This can break the immediate construction cycle, but it is normally a workaround, not the first design choice.

Another possible workaround - setter injection

Setter injection can sometimes allow bean construction to proceed before references are wired, but using it just to hide a poor dependency graph is not a good architectural fix.

Senior answer

"My first choice is to remove the circular dependency by fixing service responsibilities or extracting a third service. If the interaction is naturally asynchronous, events are another clean option. @Lazy or setter injection can break construction cycles, but I treat them as tactical workarounds because the dependency graph remains circular."

Q11) Main challenges in Blue Yonder application? Export Import? Are you holding million data which needs to be migrated in a single transaction?

Important note

Answer this using the real architecture and numbers from your project. The following is a senior-level way to structure the answer without inventing proprietary details.

Strong answer

"For large import/export or migration workloads, the challenge is not only moving the data; it is doing it without exhausting memory, keeping a database transaction open for too long, locking too much data, overflowing transaction logs, timing out, or making a failed job restart from zero. For millions of records I would normally avoid one huge transaction. I would stream/page the data, process it in bounded chunks, use batch writes, commit at controlled checkpoints, make processing idempotent, and persist job/checkpoint state so a failed job can resume."

Why NOT one transaction for a million records?

One huge transaction can create:

- long-running locks,
- large undo/transaction logs,
- high memory/resource usage,
- long-held DB connections,
- transaction timeouts,
- expensive rollback,
- impact on other users/queries,

- poor restartability.

Recommended pattern

1,000,000 records

```
-> stream/page records
-> chunk 1 (for example 1,000-5,000)
    -> validate
    -> transform
    -> batch write
    -> commit
    -> checkpoint
-> chunk 2
    -> ...
-> continue until complete
```

The exact chunk size must be load-tested; do not hard-code 5,000 as a universal number.

Failure example

Suppose 1,000,000 records are split into 200 chunks.

```
Chunks 1-126 -> SUCCESS
Chunk 127    -> FAILURE
```

A robust design should:

- retry chunk 127 if failure is transient,
- isolate/reject invalid records according to business rules,
- resume from checkpoint instead of reprocessing all prior chunks,
- avoid duplicates through idempotency.

Export considerations

- Stream from DB instead of loading all rows in heap.
- Pagination/keyset pagination/cursor based on DB capability and consistency needs.
- Stream directly to target file/object storage when possible.
- Compression if useful.
- Avoid generating one enormous in-memory byte[]/String.
- Async job model for long-running exports.
- Progress and status API.

Import considerations

- Stream file/content.
- Validate schema and business rules.
- Batch database writes.
- Separate valid vs invalid records where requirements allow.
- Idempotent import/job ID.
- Checkpoint and restart.
- Audit counts: received, validated, processed, failed, skipped.

Concurrency

Parallelism can improve throughput, but blindly increasing threads can overload:

- DB connection pool,
- database CPU/IO,
- Kafka broker,
- downstream APIs,
- memory.

Use bounded concurrency and measure throughput.

Spring Batch (if applicable)

A batch framework is useful when you need:

- chunk-oriented processing,
- transaction per chunk,
- retry/skip policies,
- job metadata,
- restartability,
- partitioning.

Do not introduce it only because data is large; use it where the operational/job model fits.

Distributed workflow

If migration touches multiple services/databases, one giant transaction is even less appropriate. Consider:

- Saga/compensation,
- transactional outbox/events,
- idempotency,
- durable workflow state.

Senior interview answer (60 seconds)

"For million-record import/export, I would avoid holding all data in memory and avoid one huge database transaction. I would stream or paginate the source, process in bounded chunks, use batch DB operations, commit per logical chunk, and maintain checkpoint state. The process should be idempotent so retries are safe. If a chunk fails because of a transient issue, retry with backoff; if data is invalid, handle it according to the business rule without losing the entire job. For long-running operations I would use an asynchronous job model and expose progress/status, with metrics and audit counts. The chunk size and degree of parallelism should be decided through performance testing and bounded by downstream capacity."

Q12) What is fault tolerance, how many types and how you used in your application?

Definition

Fault tolerance is the ability of a system to continue providing an acceptable level of service when a component, dependency, node, network call, or operation fails.

Important interview correction

There is no single official fixed number of "fault tolerance types" in Java/Spring. There are common resilience patterns. Explain the important patterns and where each is appropriate.

1. Timeout

Never wait forever for a dependency.

Example:

- Payment API normally responds in 300 ms.
- Configure a sensible connect/read/overall timeout.

Without timeout, requests can accumulate and exhaust threads/connections.

2. Retry

Retry transient failures such as temporary network errors or some 5xx responses.

Rules:

- bounded attempts,
- exponential backoff,
- jitter where appropriate,
- do not retry non-retryable business errors,
- operation should be idempotent where duplicate execution is possible.

Do not retry blindly for every exception.

3. Circuit Breaker

When a dependency is continuously failing, stop sending every request to it temporarily.

States conceptually:

```
CLOSED -> normal calls
OPEN    -> fail fast
HALF_OPEN -> allow limited test calls
```

Purpose:

- prevent cascading failure,
- reduce pressure on unhealthy dependency,
- recover automatically when service becomes healthy.

4. Bulkhead

Isolate resources so one failing dependency cannot consume all threads/connections.

Example:

- Service A calls Payment and Recommendation systems.

- Recommendation becoming slow should not consume every worker needed by Payment.

Use separate thread pools/semaphores/resource limits where appropriate.

5. Rate Limiter

Limit request rate to protect your service or respect downstream quotas.

6. Fallback

Return a safe degraded response when business requirements allow.

Example:

- Recommendation service unavailable -> return page without recommendations.

Do NOT use a fake fallback for critical payment/order consistency where incorrect data is worse than a failure.

7. Redundancy / failover

Multiple replicas, zones, nodes, DB replicas or redundant infrastructure can allow service to survive instance/node failures.

8. Queue / asynchronous buffering

Kafka/RabbitMQ can decouple producer availability from consumer processing. This does not automatically guarantee reliability; you still need retry, DLQ/DLT, idempotency, and correct offset/ack handling.

9. Graceful degradation

Keep core functionality available while optional features are degraded.

10. Idempotency

A repeated request/event should not produce duplicate side effects. Important for:

- retries,
- payment/order creation,
- Kafka redelivery,
- Saga compensation.

Spring Boot / Resilience4j style example

Conceptually:

```
@Retry(name = "inventory")
@CircuitBreaker(name = "inventory", fallbackMethod = "fallback")
public InventoryResponse getInventory(String itemId) {
    return inventoryClient.get(itemId);
}
```

In real systems also configure timeouts and monitor retry/circuit-breaker metrics.

Suggested senior answer for "How did you use it?"

Adapt to what you actually implemented:

"For external service calls, I apply timeouts first, then bounded retries only for transient failures. For repeatedly failing dependencies, a circuit breaker prevents cascading failure. I use bulkhead/concurrency limits so one downstream service cannot consume all resources. For asynchronous processing, retries must be idempotent and failed messages can move to a DLT after the retry policy is exhausted. I monitor retry count, circuit-breaker state, latency, error rate, and queue/consumer lag."

Senior warning - retry storm

If every service retries aggressively when a dependency is down, the retries can make the outage worse.

Therefore combine: Timeout + bounded Retry + Backoff/Jitter + Circuit Breaker + Bulkhead + monitoring.

Q13) Update m1 map with value of m2 using recursion.

Original input

```

Map<String,Object> m1 = {
    "k1": "v1",
    "k2": {
        "k2-1": "v2-1",
        "k2-2": "v2-2"
    }
}

Map<String,Object> m2 = {
    "k1": "v1-updated",
    "k2": {
        "k2-3": "v3-1",
        "k2-2": "v2-2-updated"
    },
    "k3": "v3"
}

```

Requirement

- Existing scalar value in m1 -> overwrite using m2.
- New key in m2 -> add to m1.
- If the value in both maps is another Map -> recursively merge it.
- Keys only in m1 -> keep them.

Recursive Java solution

```

@SuppressWarnings("unchecked")
public static void mergeMaps(Map<String, Object> target,
                             Map<String, Object> source) {

    source.forEach((key, sourceValue) -> {

        Object targetValue = target.get(key);

```

```

        if (targetValue instanceof Map<?, ?> &&
            sourceValue instanceof Map<?, ?>) {

            mergeMaps(
                (Map<String, Object>) targetValue,
                (Map<String, Object>) sourceValue
            );

        } else {
            target.put(key, sourceValue);
        }
    });
}

```

Call

```
mergeMaps(m1, m2);
```

Result

```

{
  "k1": "v1-updated",
  "k2": {
    "k2-1": "v2-1",
    "k2-2": "v2-2-updated",
    "k2-3": "v3-1"
  },
  "k3": "v3"
}

```

How to explain the recursion

"I iterate through the source map. For each key, if both the target value and source value are maps, I recursively call the same merge function on those nested maps. Otherwise, I put the source value into the target, which either inserts a new key or replaces the existing scalar/list/object value. The recursion stops naturally when the pair of values is no longer Map-vs-Map."

Complexity

For a tree-like nested map with N total entries, the traversal is roughly $O(N)$ in the number of source entries visited, assuming average $O(1)$ HashMap access.

Edge cases to mention if interviewer goes deeper

- What should happen if target has a Map but source has a String? This implementation replaces the Map with the String.
- What if values are Lists? Current implementation replaces the List; list-merge rules must be defined separately.
- What about null? Define business semantics: null can overwrite, be ignored, or mean delete depending on requirement.
- Deeply nested malicious input could cause stack-depth issues; for normal configuration/JSON-like business objects recursion is often acceptable, but an iterative approach is possible if depth is unbounded.
- If source nested maps must not be shared by reference after merge, perform defensive

copying rather than directly putting mutable objects.

FINAL PREPARATION - WHAT TO STUDY MOST

Based on this interview set, prepare these areas deeply:

1. Transaction management

- @Transactional behavior.
- programmatic transactions.
- rollback rules.
- transaction propagation/isolation.
- distributed transaction vs Saga.
- idempotency/outbox.

2. Java concurrency

- Thread/ExecutorService.
- ThreadPoolExecutor sizing and queues.
- CompletableFuture.
- Virtual Threads.
- java.util.concurrent coordination utilities.
- thread safety and race conditions.

3. Spring internals

- dependency injection.
- @Primary/@Qualifier.
- ApplicationContext.
- bean lifecycle.
- BeanFactoryPostProcessor vs BeanPostProcessor.
- proxies/AOP.
- circular dependency.

4. Modern Java

- Java 8, 11, 17, 21 and 25 LTS generations.
- know that Java 27 is latest as of October 2026.
- go deep on features actually used.

5. Reliability and scale

- timeout/retry/circuit breaker/bulkhead.
- idempotency.
- large dataset streaming/batching.
- backpressure.
- checkpoint/restart.
- monitoring/observability.

QUICK INTERVIEW RULE FOR A 9-10 YEAR PROFILE

For each answer use this structure:

1. Define it in 1-2 sentences.
2. Explain how it works.
3. Give one production example.
4. Explain one failure/edge case.
5. Mention a trade-off/alternative.

Example: "CompletableFuture is for asynchronous/composable tasks. I use it when independent calls can run concurrently. I use a controlled executor rather than blindly depending on the common pool. I handle timeout and exceptions explicitly. For a modern blocking I/O service on Java 21+, I would also compare it with Virtual Threads because they may produce simpler imperative code."

Current Java reference note (verified October 2026)

Oracle currently lists Java 27 as the latest Java SE release and Java 25 as the latest LTS. Java 27 was released in September 2026. Official references:

- Oracle Java downloads: <https://www.oracle.com/java/technologies/downloads/>
- Oracle Java SE support roadmap:
<https://www.oracle.com/java/technologies/java-se-support-roadmap.html>
- Oracle JDK 27 release notes:
<https://www.oracle.com/java/technologies/javase/27all-relnotes.html>